HDOC DAY-51

Next Greater Element Using Pointers and Nested Loops

Student Name	Dhruv Pandey
SAP ID	590037660
Batch	15
Program	B.Tech CSE
Language	C

Activity: Write a C program to find the nearest next greater element on the right of every array element. Pass the array to **next_greater()** using its address and use pointers with nested loops.

1. Aim

To write and execute a C program that reads a variable-length integer array and finds the nearest greater element to the right of each element by passing the array address to a function using pointers and using nested loops.

2. Problem Requirements

1. Read the array size and elements from the user.
2. Pass the array to next_greater() through its address (pointer).
3. For each element, search only toward its right.
4. Choose the first/nearest element that is greater than the current element.
5. Print -1 when no greater element exists to the right.
6. Display answers in order, separated by commas.
7. Use nested loops.

3. Algorithm

1. Start.
2. Input n, the number of elements.
3. Declare a variable-length array arr[n].
4. Input all n array elements.
5. Call next_greater(arr, n). Here arr gives the address of the first element.
6. Inside next_greater(), use an outer loop for i = 0 to n-1.
7. Set next = -1 for the current element.
8. Use an inner loop for j = i+1 to n-1.
9. If *(arr+j) > *(arr+i), store *(arr+j) in next and break because it is the nearest greater element.
10. Print next with comma formatting.
11. Stop.

4. C Program

```c
#include <stdio.h>

void next_greater(int *arr, int n)
{
    for (int i = 0; i < n; i++)
    {
        int next = -1;

        for (int j = i + 1; j < n; j++)
        {
            if (*(arr + j) > *(arr + i))
            {
                next = *(arr + j);
                break;
            }
        }

        printf("%d", next);
        if (i < n - 1)
            printf(", ");
    }
    printf("\n");
}

int main(void)
{
    int n;

    printf("Enter the size of array: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Array size must be positive.\n");
        return 0;
    }

    int arr[n];

    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("Next greater elements: ");
    next_greater(arr, n);

    return 0;
}
```

5. How the Pointer Passing Works

In main(), the call **next_greater(arr, n)** passes the address of the first array element. The parameter **int *arr** receives that address. Therefore, ***(arr + i)** accesses the same value as **arr[i]**. No separate copy of the full array is made.

6. Sample Execution / Test Cases

Example 1	
Input size	4
Array elements	4 5 2 25

Program output	5, 25, 25, -1

Example 2	
Input size	4
Array elements	13 7 6 12
Program output	-1, 12, 12, -1

Example 3	
Input size	4
Array elements	1 3 2 4
Program output	3, 4, 4, -1

Example 4	
Input size	5
Array elements	1 2 3 4 5
Program output	2, 3, 4, 5, -1

7. Dry Run for Example 1

Current element	Search on right	Nearest greater
4	5, 2, 25	5
5	2, 25	25
2	25	25
25	No element on right	-1

8. Complexity

Time Complexity: O(n^2) in the worst case because nested loops may compare each element with many elements to its right.
Extra Space: O(1), apart from the input array, because only a few variables are used.

9. Result

The program successfully finds and displays the nearest next greater element to the right for every array element using pointers and nested loops. If no greater element exists, it displays -1.